SALESFORCE ADMIN

HANDS-ON PROJECT

Hospital Management System

Implement everything yourself — no answers given

Org	Developer Edition or Sandbox
Duration	6 to 8 Hours (self-paced)
Style	Each task gives you the REQUIREMENT only. You decide the field types, formulas, and flow logic.
Rule	Do NOT look at the answer key until you have attempted the task.

■ You are the Salesforce Admin. You decide field types, formulas, validation logic, and flow design. The 'Think about this' boxes are hints — not answers.

Phase 1 — Data Model

TASK 1.1 | Custom Objects

Create the Appointment custom object

Context: City Care Hospital needs to track patient appointments inside Salesforce. This is a new concept — it does not exist as a standard object.

Requirements:

- The object should be called 'Appointment' with API name Appointment__c.
- Each record should auto-generate a unique name like APT-0001, APT-0002 (you decide which field type does this).
- Users should be able to run reports on this object.
- Activities (Tasks, Emails) should be enabled on this object.
- The object should be visible to users immediately after creation.
- Field history tracking should be on.

Think about this:
■ What deployment status makes an object available to users?
■ What Record Name type generates APT-0001 style names automatically?

TASK 1.2 | Custom Objects

Create the Medical Record custom object

Context: Each appointment may have one or more medical records attached (diagnosis, prescriptions). This also needs to be a new custom object.

Requirements:

- Object Label: Medical Record. API Name: Medical_Record__c.
- Auto-number naming: MR-0001, MR-0002 etc.
- Reports and field history must be enabled.
- Deployed status.

Think about this:
■ Is there any standard object in Salesforce that already handles medical records? (Answer: No — you must create custom.)

TASK 1.3 | Custom Fields

Add fields to Appointment__c — you decide the field types

Context: The hospital wants to store the following information on each appointment. Your job is to decide the correct Salesforce field type for each piece of information and create it.

Requirements:

- The date on which the appointment is scheduled. (What field type stores a calendar date?)
- The time of the appointment, e.g. '10:30 AM'. (Which text-based field is appropriate?)
- The current status of the appointment — it can be: Scheduled, Completed, Cancelled, or No Show. Default should be Scheduled. (Which field type gives a fixed list of choices?)
- A description of why the patient is visiting. Should support long text. (Which field allows long paragraphs?)
- Urgency level: High, Medium, or Low. Default Medium.
- A yes/no flag indicating if a follow-up visit is needed.
- The fee charged for the consultation — should support decimals and currency symbol.
- The hospital department: Cardiology, Orthopedics, General, Pediatrics, Neurology.

Think about this:

- ***Text vs Text Area vs Text Area (Long) — when do you use each?***
- ***Currency vs Number vs Percent — what is the difference?***
- ***Picklist vs Multi-Select Picklist — when does a patient have ONE department vs MULTIPLE?***
- ***What is the difference between a Checkbox and a Picklist with Yes/No values?***

TASK 1.4 | Custom Fields

Add fields to Medical_Record__c and Contact (Patient)

Context: Medical records need clinical data. Patient Contact records also need some extra fields.

Requirements:

- Medical_Record__c: A diagnosis field — long descriptive text, required.
- Medical_Record__c: A prescription field — very long text.
- Medical_Record__c: Blood pressure reading, e.g. '120/80' — short text.
- Medical_Record__c: Body temperature in Fahrenheit — decimal number.
- Medical_Record__c: Patient weight in kg — decimal number.
- Medical_Record__c: Date for follow-up visit — optional.
- Medical_Record__c: Status of the record — Draft, Finalized, or Archived. Default: Draft.
- Contact (standard object): Patient blood type — A+, A-, B+, B-, AB+, AB-, O+, O-.
- Contact: Patient date of birth.
- Contact: Patient ID — auto number format PAT-0001.

Think about this:
■ Can you add custom fields to standard objects like Contact? (Yes — go to Object Manager > Contact > Fields & Relationships > New)
■ Which fields should be marked Required — and which should not?

Phase 2 — Relationships

TASK 2.1 | Lookup Relationship

Link Appointment to Doctor

Context: Each appointment is with a Doctor. In this org, Doctors are stored as Contact records. The appointment should know which Doctor it is for.

Requirements:

- Create a relationship field on Appointment__c that links to Contact (the Doctor).
- If the Doctor's Contact record is deleted, the appointment should NOT be deleted — it should just lose the link.
- The field label should be 'Doctor'.
- The related list on the Contact record should show all appointments for that doctor.

Think about this:
■ Lookup vs Master-Detail — which one allows the child to survive if the parent is deleted?
■ On which object does the relationship field sit — Appointment__c or Contact?

TASK 2.2 | Lookup Relationship

Link Appointment to Patient

Context: Patients are also stored as Contact records (a different Contact from the Doctor). Each appointment belongs to one patient.

Requirements:

- Create another Lookup from Appointment__c to Contact for the patient.
- Field label: Patient.
- The related list name on Contact should say 'Patient Appointments'.

Think about this:
■ Can one object (Appointment__c) have two Lookups to the same object (Contact)? (Yes — one for Doctor, one for Patient.)

TASK 2.3 | Master-Detail Relationship

Link Medical Record to Appointment

Context: A Medical Record only makes sense if an Appointment exists. If an Appointment is deleted, all its Medical Records should also be deleted automatically. The hospital also wants to see a count of medical records directly on the Appointment record.

Requirements:

- Create a relationship field on Medical_Record__c linking to Appointment__c.
- The relationship must be the type that: (a) makes the parent required, (b) deletes children when parent is deleted, (c) allows Roll-Up Summary fields on the parent.
- Relationship field label: Appointment.
- Add the Medical Records related list to the Appointment page layout.

Think about this:
■ Which relationship type cascades delete from parent to child?
■ Which relationship type enables Roll-Up Summary fields on the parent object?
■ Where does the relationship field physically sit — on the parent or the child?

Phase 3 — Formula Fields & Roll-Up Summary

TASK 3.1 | Formula Fields

Create a 'Days Since Appointment' field on Appointment__c

Context: Management wants to see, at a glance, how many days ago each appointment took place. This should update automatically every day without anyone editing the record.

Requirements:

- Field name: Days Since Appointment. API: Days_Since_Appointment__c.
- Should return a whole number (no decimals).
- Should calculate: today's date MINUS the appointment date.
- If the Appointment_Date__c field is blank, the field should return 0 instead of an error.
- No one should be able to manually edit this field.

Think about this:
- ***Which field type calculates automatically and is always read-only?***
- ***Which Salesforce function gives you today's date?***
- ***Which function checks if a field is empty/blank?***
- ***How do you handle null/blank dates so the formula doesn't break?***

TASK 3.2 | Formula Fields

Create an 'Is Overdue' checkbox formula on Appointment__c

Context: The team wants a checkbox that is automatically checked (TRUE) when an appointment is overdue — meaning the date has passed but it was never completed or cancelled.

Requirements:

- Field name: Is Overdue. API: Is_Overdue__c. Return type: Checkbox.
- Should be TRUE when: the Appointment Date is before today, AND Status is not 'Completed', AND Status is not 'Cancelled'.
- Should be FALSE in all other cases.

Think about this:
- ***How do you compare a date field against today's date in a formula?***
- ***Which logical function combines multiple conditions where ALL must be true?***
- ***Picklist fields cannot be compared with = in formulas — which special function is used instead?***

TASK 3.3 | Formula Fields

Create an 'Appointment Summary' text formula on Appointment__c

Context: For a quick read in list views, management wants one field that shows the status and priority together, like: 'Scheduled - High Priority'.

Requirements:

- Field name: Appointment Summary. API: Appointment_Summary__c. Return type: Text.
- Output format should be: [Status] - [Priority] Priority
- Example: if Status = Scheduled and Priority = High, output = 'Scheduled - High Priority'

Think about this:
■ How do you join/concatenate text values in a Salesforce formula?
■ Can you directly reference picklist fields in text formulas? (Yes — TEXT() function may be needed in some contexts.)

TASK 3.4 | Roll-Up Summary Fields

Add Roll-Up Summary fields on Appointment__c

Context: The hospital wants to see — directly on each Appointment record — how many medical records are attached, and how many of those are in 'Finalized' status.

Requirements:

- Field 1: 'Total Medical Records' — count ALL Medical_Record__c records linked to this appointment.
- Field 2: 'Total Finalized Records' — count only those Medical_Record__c records where Record_Status__c = 'Finalized'.
- Both fields should update automatically when Medical Records are added or removed.

Think about this:
■ Which relationship type is required before you can create Roll-Up Summary fields?
■ Roll-Up Summary fields are created on which object — the master or the detail?
■ For Field 2 — where do you add the filter condition in the Roll-Up field wizard?

Phase 4 — Record Types & Page Layouts

TASK 4.1 | Record Types

Create two Record Types on Appointment__c

Context: The hospital has two types of appointments: patients who come in person (In-Person Visit) and patients who connect via video call (Telemedicine). These two types need different fields visible on the form.

Requirements:

- Record Type 1: Label = In-Person Visit. Active = Yes. Assign to all profiles.
- Record Type 2: Label = Telemedicine. Active = Yes. Assign to all profiles.
- Both record types should be selectable when creating a new Appointment.

Think about this:
■ Where in Setup do you create Record Types — at the org level or inside Object Manager?
■ What happens if you don't assign a Record Type to a profile?

TASK 4.2 | Page Layouts

Create two different Page Layouts for the two Record Types

Context: In-Person appointments need to show Department and Consultation Fee. Telemedicine appointments do not need Department since it is online. Design the layouts appropriately.

Requirements:

- Layout 1 (In-Person Visit Layout): Include all fields — Doctor, Patient, Date, Time, Department, Status, Priority, Reason, Consultation Fee, Follow Up Required, Days Since Appointment, Is Overdue.
- Layout 2 (Telemedicine Layout): Include — Doctor, Patient, Date, Time, Status, Priority, Reason, Follow Up Required, Days Since Appointment. Do NOT show Department or Consultation Fee.
- Map Layout 1 to the In-Person Visit Record Type.
- Map Layout 2 to the Telemedicine Record Type.
- Both layouts must have the Medical Records related list at the bottom.

Think about this:
■ If you hide a field from the Page Layout, can the user still see it in reports? (Yes — Page Layout only controls the record page view, not reports.)
■ Where do you map a Record Type to a Page Layout — in the Record Type settings or in the Profile settings?

Phase 5 — Profiles & Field-Level Security

TASK 5.1 | Profiles

Create two Custom Profiles

Context: Two types of users will use this system. Doctors should be able to manage appointments and medical records but NOT delete them. Receptionists manage appointments and patient contacts but cannot touch medical records.

Requirements:

- Profile 1 — HMS Doctor Profile: On Appointment__c: can Read, Create, Edit. Cannot Delete. On Medical_Record__c: can Read, Create, Edit. Cannot Delete. On Contact: Read only. On Account: Read only.
- Profile 2 — HMS Receptionist Profile: On Appointment__c: can Read, Create, Edit, Delete. On Contact: can Read, Create, Edit. Cannot Delete. On Medical_Record__c: Read only. Cannot create or edit.
- Both profiles must be Custom Profiles — clone from Standard User.

Think about this:
- ***Why can't you just use a Standard Profile and tweak it?***
- ***What is the difference between 'Read' and 'View All' on an object?***

TASK 5.2 | Field-Level Security (FLS)

Restrict access to the Consultation Fee field

Context: The Consultation_Fee__c field contains financial information. Doctors should never see this field. Only Receptionists and Admins should see and edit it.

Requirements:

- For HMS Doctor Profile: Consultation_Fee__c should be completely hidden — not visible at all.
- For HMS Receptionist Profile: Consultation_Fee__c should be visible and editable.
- For System Administrator: visible and editable.
- Test: log in as a Doctor-profile user and confirm the field does not appear on the Appointment record, even if it is on the Page Layout.

Think about this:
- ***If a field is on the Page Layout but FLS is Hidden — what does the user see?***
- ***Where do you set FLS — in the Profile settings or in the field's own settings (or both ways work)?***

Phase 6 — OWD, Role Hierarchy, Public Groups & Sharing Rules

TASK 6.1 | OWD

Set Organisation-Wide Defaults

Context: By default, appointments and patient data should be private — only the record owner and their managers should see records. You will open access selectively through sharing rules.

Requirements:

- Appointment__c: Set OWD so that by default only the record owner can see their own appointments.
- Medical_Record__c: Set OWD so that Medical Records automatically follow the sharing of their parent Appointment.
- Contact: Set OWD so that patient records are private by default.

Think about this:
- ***Which OWD setting = only the owner can see/edit?***
- ***Which OWD setting on Medical_Record__c means 'inherit from parent Appointment'?***
- ***After setting OWD to Private, will managers in the role hierarchy still see records? Why?***

TASK 6.2 | Role Hierarchy

Build the hospital role hierarchy

Context: The hospital has a management chain. Senior managers should automatically see all appointments owned by staff below them.

Requirements:

- Create this hierarchy (top to bottom): Chief Medical Officer at the top. Department Head reports to Chief Medical Officer. Doctor reports to Department Head. Receptionist reports to Department Head.
- Assign at least one test user to each role.
- Verify: a Doctor's appointment record — can the Department Head see it without any sharing rule? Can a Receptionist at the same level see it?

Think about this:
- ***Does Role Hierarchy grant access upward or downward?***
- ***Can a user at the same level (peer) see another user's private record via role hierarchy?***

TASK 6.3 | Public Groups

Create two Public Groups

Context: You need to share appointments with groups of users that don't fit neatly into one role. Public Groups let you combine users and roles together.

Requirements:

- Group 1 — 'All Doctors': add the entire Doctor role as a member.
- Group 2 — 'Emergency Team': add the Doctor role AND the Department Head role as members.

Think about this:
- ***Can a Public Group contain both individual users AND entire roles?***
- ***If a new user joins the Doctor role later, do they automatically get access through the group?***

TASK 6.4 | Criteria-Based Sharing Rules

Create two Criteria-Based Sharing Rules on Appointment__c

Context: Now that OWD is Private, you need to open up access selectively based on conditions — not manually record by record.

Requirements:

- Rule 1: Any appointment where Priority = High should be shared with the Emergency Team group with Read/Write access.
- Rule 2: Any appointment where Status is NOT equal to Cancelled should be shared with All Doctors group with Read Only access.
- Both rules must be active.
- Test Rule 1: Create an appointment with Priority = Medium. Check Emergency Team access. Then change it to High. Does Emergency Team now have access automatically?

Think about this:
■ What is the difference between an Owner-Based Sharing Rule and a Criteria-Based Sharing Rule?
■ If an Appointment's Priority changes from High to Low, does the sharing from Rule 1 get automatically removed?

Phase 7 — Matching Rules, Duplicate Rules & Validation Rules

TASK 7.1 | Matching Rules

Create a Matching Rule to detect duplicate patients

Context: The hospital keeps getting duplicate patient records when receptionists enter the same person twice. You need Salesforce to detect when two Contact records have the same First Name, Last Name, and Date of Birth.

Requirements:

- Object: Contact.
- Match on: First Name (Exact), Last Name (Exact), Date_of_Birth__c (Exact).
- ALL three fields must match for Salesforce to flag it as a duplicate.
- Activate the Matching Rule.

Think about this:
■ What is the difference between Exact match and Fuzzy match — when would you use Fuzzy on a Name field?
■ Can you activate a Duplicate Rule before the Matching Rule is active?

TASK 7.2 | Duplicate Rules

Create a Duplicate Rule for Contact

Context: Now that the Matching Rule can detect duplicates, you need to define WHAT Salesforce does when a duplicate is found — block it or just warn.

Requirements:

- When a new Contact is created that matches an existing one: show the user a warning but allow them to save if they choose.
- When an existing Contact is edited to match another: completely block the save.
- Activate the Duplicate Rule.
- Test: try to create a Contact with the same name and DOB as an existing one. What do you see?

Think about this:
■ Alert vs Block — what is the practical difference from the user's perspective?
■ Why might you choose Alert on Create but Block on Edit?

TASK 7.3 | Validation Rules

Build 4 Validation Rules on Appointment__c — requirements only, you write the formula

Context: The admin has given you the business rules below. You must translate each into a Salesforce Validation Rule formula. Remember: the formula should return TRUE when you want to BLOCK the save.

Requirements:

- Rule 1 — No Past Scheduling: When someone creates a brand new appointment, the Appointment Date cannot be in the past. (Only applies on create, not on edit.)
- Rule 2 — Completed Needs a Date: An appointment cannot be saved with Status = Completed if the Appointment Date field is blank.
- Rule 3 — High Priority Needs a Reason: If Priority is set to High, the Reason for Visit field must not be empty.
- Rule 4 — Cannot Reopen Cancelled: Once an appointment is Cancelled, you cannot change it back to Scheduled. (Think: what was the previous value of Status?)

Think about this:
■ Rule 1: Which function tells you if the record is NEW vs being edited? How do you compare a date against today?
■ Rule 2: Which function checks if a field is empty? How do you compare a picklist value in a formula?
■ Rule 3: Same as Rule 2 but combined with a blank check on a text field.
■ Rule 4: Which function gives you the value a field had BEFORE the current save? How do you check if the current save is NOT a new record?

Phase 8 — Screen Flow: Book New Appointment

TASK 8 | Screen Flow (All 7 concepts)

Build a Screen Flow to book an appointment from the Patient's Contact page

Context: Receptionists currently create appointments manually. You need to build a guided Screen Flow that a receptionist can launch from the Patient's Contact record. The flow walks them through booking — showing the patient's name, collecting appointment details, confirming, and then creating the appointment.

Requirements:

- CONCEPT 1 — Input Variable: The flow must receive the Patient Contact's record ID from the page it is launched from. (What variable type, what setting makes it receivable from outside?)
- CONCEPT 2 — Display Record Information: The first screen should show the patient's full name so the receptionist can confirm they are booking for the right person. You do not have this data yet — you only have the record ID. (What element fetches the record data?)
- CONCEPT 3 — Screen Components: Screen 1 should collect: Appointment Date (required), Appointment Time, Department (a choice from the picklist values), Priority (High/Medium/Low), and Reason for Visit (required long text). Each input must be mapped to a variable.
- CONCEPT 4 — Create Child Records: After the confirmation screen, the flow must create a new Appointment__c record using all the values collected. The Patient__c field must be set to the Contact ID received at the start.
- CONCEPT 5 — Update Existing Records: After creating the appointment, update the Contact record (the Patient) — set a custom field Last_Appointment_Date__c to the appointment date the user entered. (Add this Date field to Contact first.)
- CONCEPT 6 — Passing Record ID: The flow must receive the Contact record ID automatically when launched — the receptionist should NOT have to type it. (How does the page pass its record ID into the flow? What must the variable be named and what setting must be ON?)
- CONCEPT 7 — Launching from Button: Add a Quick Action button called 'Book Appointment' on the Contact page layout so receptionists can launch this flow with one click.

Think about this:
■ What is the variable name convention Salesforce uses to auto-receive a record ID from a record page?
■ Get Records element — if no record is found, what does the output variable contain? Should you check for this?
■ Screen components — what is the difference between a Display Text component and an Input component?
■ Create Records — how do you tell it to create one specific record (not a collection)?
■ Where exactly in App Builder or Quick Action setup do you connect the page's record ID to the flow's input variable?

Phase 9 — Record-Triggered Flows

TASK 9.1 | Before-Save Flow + Decision + Assignment

Auto-set Priority based on Department when an Appointment is created or updated

Context: The medical team says Cardiology and Neurology appointments are always critical. Whenever an appointment is saved with one of these departments, Priority should automatically become 'High' — without any receptionist having to set it manually.

Requirements:

- Trigger: when an Appointment is created OR updated, BEFORE the record is saved to the database.
- If Department is Cardiology or Neurology: set Priority to High automatically.
- If Department is anything else: leave Priority as-is.
- This must NOT cause a second/separate save operation — it must happen as part of the original save.
- Use a Decision element to check the Department value.
- Use an Assignment element to set the Priority field on the record.

Think about this:
■ Before-Save vs After-Save — which one lets you change the record being saved WITHOUT a second DML?
■ In a Before-Save flow, how do you reference the record being saved? (Hint: it is a special global variable.)
■ In the Assignment element, do you use Update Records or just set the field directly? (Think: Before-Save flows cannot do DML on any record — but assignment to $Record is allowed.)

TASK 9.2 | After-Save Flow + Get Records + Create Records

Create a follow-up Task when an Appointment is marked Completed

Context: When a receptionist marks an appointment as Completed, a Task should automatically be created and assigned to the Doctor to remind them to follow up with the patient in 7 days.

Requirements:

- Trigger: when an Appointment is updated, AFTER the record is saved.
- Only fire when Status changes TO 'Completed' — not every time the record is saved. (How do you check what the previous value of Status was?)
- Fetch the Doctor's Contact record using the Doctor__c field on the appointment.
- If no Doctor is linked to the appointment, the flow should end without creating a task (no error).
- Create a Task: Subject = 'Follow up with patient', Due Date = 7 days from today, Assigned To = the Doctor, Related To = the Appointment.

Think about this:
■ Entry conditions: how do you make the flow only run when Status changes TO Completed, not on every save?
■ Get Records: what object are you querying? What is the condition?
■ After Get Records: what should you check before trying to use the fetched record?
■ Create Records: Task's ActivityDate needs to be 7 days from today — how do you calculate a date + 7 in a flow (Hint: Formula resource)?

TASK 9.3 | After-Save Flow + Get Records + Update Records

Copy Patient Blood Type to Appointment when it is created

Context: Doctors want to see the patient's blood type directly on the Appointment record without opening the Contact. When a new Appointment is created and linked to a Patient, automatically copy the Patient's Blood_Type__c into a field on the Appointment.

Requirements:

- First create a new Text field on Appointment__c: Patient_Blood_Type__c.
- Trigger: when an Appointment is CREATED (not updated), After Save.
- Only run if the Patient__c field is not blank.
- Fetch the Patient's Contact record using the Patient__c field.
- If the Contact is not found, end the flow gracefully.
- Update the Appointment record — set Patient_Blood_Type__c = the value from the Contact's Blood_Type__c field.

Think about this:
■ After-Save flows cannot modify the triggering record using an Assignment element — you must use Update Records. Why is Before-Save not suitable here? (Hint: you need to query another object first.)
■ Get Records returns null if nothing is found — what element checks for this?
■ Update Records — to update the SAME Appointment that triggered the flow, what condition do you use to identify it?

Verification Checklist — tick after completing each task

☐	Custom object Appointment__c exists, deployed, with auto-number name APT-0001
☐	Custom object Medical_Record__c exists with auto-number MR-0001
☐	All 8 custom fields exist on Appointment__c with correct field types
☐	Custom fields exist on Medical_Record__c and Contact
☐	Lookup Doctor__c (Appointment → Contact) exists
☐	Lookup Patient__c (Appointment → Contact) exists
☐	Master-Detail Appointment__c (on Medical_Record__c) exists
☐	Days_Since_Appointment__c formula returns correct number
☐	Is_Overdue__c shows TRUE for past-dated open appointments
☐	Appointment_Summary__c shows "Scheduled - High Priority" style text
☐	Total_Medical_Records__c Roll-Up counts correctly
☐	Total_Finalized_Records__c only counts Finalized medical records
☐	Two Record Types exist: In-Person Visit and Telemedicine
☐	Two Page Layouts exist and are mapped to the correct Record Types
☐	HMS Doctor Profile created with correct object permissions
☐	HMS Receptionist Profile created with correct object permissions
☐	Consultation_Fee__c is HIDDEN for Doctor profile — verified by logging in
☐	OWD set: Appointment__c = Private, Medical_Record__c = Controlled by Parent
☐	Role Hierarchy: CMO > Dept Head > Doctor/Receptionist
☐	Public Group All_Doctors contains Doctor role
☐	Public Group Emergency_Team contains Doctor + Dept Head roles
☐	Sharing Rule 1: High Priority appointments shared with Emergency Team
☐	Sharing Rule 2: Non-cancelled appointments shared with All Doctors
☐	Matching Rule on Contact: First Name + Last Name + DOB (all Exact) — Active
☐	Duplicate Rule on Contact: Alert on Create, Block on Edit — Active
☐	Validation Rule 1: Cannot create appointment with past date
☐	Validation Rule 2: Completed status requires Appointment Date
☐	Validation Rule 3: High Priority requires Reason for Visit
☐	Validation Rule 4: Cannot change Cancelled back to Scheduled

■	Screen Flow: launched from Contact page, shows patient name, creates Appointment
■	Screen Flow: updates Last_Appointment_Date__c on Contact after booking
■	Before-Save Flow: Cardiology/Neurology appointments auto-set Priority to High
■	After-Save Flow: Task created when Appointment changes to Completed
■	After-Save Flow: Patient blood type copied to Appointment on create